AI335L Deep Learning Lab

Experiment No. 11

Deadline: May 17, 2026 12:00 AM

Lab Task (Phase 4)

Refinement: Transfer Learning, Generation, and Hyperparameter Search

1. From Working Model to Better Model

By the end of last week you had a deep architecture that trained, beat (or at least competed with) your baseline, and showed you something interesting when you visualized it. This week is about making it meaningfully better, in three directions at once: smarter use of pretrained knowledge through transfer learning, a generative component that adds capability your discriminative model alone cannot provide, and a principled hyperparameter search that replaces guesswork with evidence.

This is the heaviest technical week of the project. Three workstreams in seven days is a lot, and you will not finish all of them at the same depth. That is fine. Plan parallel work across team members from day one, and treat the report as an honest account of what you tried, what worked, and what you ran out of time to explore.

2. Learning Objectives

By the end of Phase 4, you will be able to:

- Apply transfer learning beyond "load pretrained weights and train", including layer freezing strategies, differential learning rates, and gradual unfreezing
- Implement a generative model (Autoencoder, Variational Autoencoder, or GAN) and integrate it meaningfully with your main project
- Evaluate generative models with appropriate metrics (reconstruction loss, KL divergence, FID, perceptual quality)
- Design and execute a systematic hyperparameter search using Optuna, Ray Tune, or equivalent
- Read HPO results critically: identify which hyperparameters actually matter, recognize spurious wins, allocate compute efficiently
- Diagnose and address training pathologies specific to generative models (mode collapse, posterior collapse, discriminator dominance)

3. The Three Workstreams

Phase 4 has three concurrent workstreams. They are not independent: results from each feed into the report and into the final model you take into Phase 5. Plan ownership before Friday.

Workstream	Goal	Effort Allocation
Transfer Learning	Replace or augment your Phase 3 backbone with a pretrained model used in a disciplined way. Compare strategies (feature extraction vs full fine-tuning vs differential LR).	≈ 30% of week
Generative Component	Build an Autoencoder, VAE, or GAN that solves a real sub-problem in your project (data augmentation, anomaly detection, denoising, synthesis, or compression).	≈ 40% of week
Hyperparameter Search	Run a principled HPO study on your best discriminative model, identifying which hyperparameters matter and what their useful ranges are.	≈ 30% of week

Coordination tip: if you have three team members, give each a primary workstream. If two members, pair Transfer Learning with HPO (they share infrastructure) and one person owns the generative component.

4. What You Will Submit

Five artifacts, all linked from your repository:

Deliverable	Description	Format
Updated Repository	Transfer learning module, generative model module, HPO scripts, all under src/.	GitHub URL + tag
Transfer Learning Results	At least three configurations compared with metrics and curves.	Tables + plots
Generative Model + Samples	Trained generator/decoder weights plus a notebook or script that produces sample outputs.	.pt/.h5 + samples/

Deliverable	Description	Format
HPO Study	Optuna study database (or equivalent), with at least 20 completed trials and analysis plots.	.db / .pkl + plots
Refinement Report	Written report covering all three workstreams plus integration discussion.	PDF, 12–16 pages

5. Detailed Task Breakdown

Task 1: Transfer Learning Done Properly

Phase 3 may have used a pretrained backbone. Phase 4 makes that use rigorous. You will run and compare at least three transfer learning configurations on the same task.

Configuration	What It Means
Feature extraction	Freeze the entire pretrained backbone. Train only a new classification (or regression) head on top. Cheapest option, often a strong baseline.
Full fine-tuning	Unfreeze everything and train end-to-end with a small learning rate. Highest capacity to adapt; risks catastrophic forgetting.
Differential learning rates	Train the head with a normal LR (e.g., 1e-3) and the backbone with 10–100x smaller (e.g., 1e-5). Often the best of both worlds.

Required:

- Document the source model: name, version, where it was pretrained, dataset, and license
- Report parameter counts: total, frozen, trainable for each configuration
- Same train/val/test split, same metrics, same number of epochs (or same compute budget) across configs for a fair comparison
- At least one experiment with gradual unfreezing: start with frozen backbone, unfreeze layer-by-layer (or block-by-block) over training
- Plot validation curves of all configurations on the same axes

Watch for catastrophic forgetting: if your fully-fine-tuned model performs worse than feature-extraction at the end, the backbone has lost useful pretrained knowledge. Lower the backbone learning rate, freeze the lower layers, or use a warmup phase that trains only the head first.

Task 2: Generative Component

Pick one generative model and integrate it into your project. “Integrate” is the operative word. A standalone GAN that has nothing to do with your main task does not earn full marks. The generative component must serve the project.

Choosing the Right Model

Model	Strengths	Use If Your Project Needs
Autoencoder	Simple, stable, fast. Learns meaningful compressed representations.	Anomaly detection, denoising, dimensionality reduction, pretraining feature extractors.
VAE	Probabilistic, smooth latent space, can sample new data. More stable to train than GANs.	Synthetic data generation, latent-space interpolation, controlled generation, augmentation for small classes.
GAN	Sharp, realistic samples in image and audio domains. Strong perceptual quality.	High-fidelity synthesis, style transfer, image-to-image translation, super-resolution.

Required for Every Generative Track

- Architecture diagram of the generator/decoder and (if applicable) discriminator
- Loss function with each term named and explained (reconstruction, KL, adversarial, etc.)
- Training curves for all loss components, not just total loss
- At least 16 sample outputs in a grid, presented in the report
- Quantitative evaluation appropriate to the model (reconstruction MSE for AE; reconstruction + KL for VAE; FID or Inception Score for GANs on images)
- An integration paragraph: how does this generative model improve the main project? Quantify the improvement if possible.

Track-Specific Requirements

Autoencoder track:

- Compare a vanilla AE with at least one variant (Denoising AE, Sparse AE, or Convolutional AE depending on data type)
- Visualize the latent space using PCA or t-SNE colored by class
- If used for anomaly detection: report ROC-AUC of reconstruction error as the anomaly score

VAE track:

- Implement the reparameterization trick correctly: sample from N(0,1), scale by sigma, shift by mu

- Report both reconstruction loss and KL divergence separately, on every plot
- Address posterior collapse if it appears (KL drops to 0 and the decoder ignores z): use KL annealing, free bits, or beta-VAE
- Show latent-space interpolations between two real samples

GAN track:

- Use a stable variant unless you have a strong reason: DCGAN, WGAN-GP, or LSGAN are reasonable starting points
- Track and address mode collapse (generator produces only a few distinct outputs). Diagnose with per-class sample counts or clustering of generated samples.
- Track and address discriminator dominance (D loss → 0, G loss exploding). Reduce D learning rate, add gradient penalty, or use spectral normalization.
- Compute FID on at least 1,000 generated samples vs 1,000 real samples (only if your data is image-like)

Task 3: Hyperparameter Search Pipeline

Random tweaking and "well, this seemed to work" are not hyperparameter tuning. This week you build a real search pipeline.

Required:

- Use a real HPO library: Optuna (recommended), Ray Tune, Weights & Biases Sweeps, or Keras Tuner
- Define a config-driven search space in a single file (search_space.yaml or similar)
- Use a search strategy beyond grid search: Bayesian (TPE), Hyperband, or BOHB. Grid search alone is rejected for budgets above ~20 trials.
- Use early stopping (Optuna pruners, ASHA, etc.) to kill bad trials before they consume their full budget
- Persist the study so it can be resumed: SQLite for Optuna, ray.tune.run with checkpoint_dir, or equivalent
- Log every trial: hyperparameters, final metric, training time, and (for failed trials) the failure reason

What to Tune

Pick at most 6–8 hyperparameters. More than that and your budget per hyperparameter becomes too small to learn anything. Reasonable picks:

- Learning rate (almost always the most important; search log-scale, e.g., 1e-5 to 1e-1)
- Batch size (powers of 2, but be aware it interacts with learning rate)
- Weight decay (log-scale, 1e-6 to 1e-2)
- Dropout rate (linear, 0.0 to 0.6)
- Architecture-specific: depth, width, number of attention heads
- Optimizer choice (AdamW, SGD+momentum, RMSprop) as a categorical hyperparameter

Task 4: HPO Execution

Run the search. Minimum requirements:

- At least 20 completed trials, more if your training is fast enough
- All trials seeded for reproducibility (the seed itself can be a tuned hyperparameter or fixed)
- Total compute budget documented in the report (GPU-hours or wall-clock time)
- If a trial fails, capture the error and exclude it from analysis (do not silently drop)

Search hygiene: never tune on the test set. Use validation for the HPO objective. The test set is checked exactly once at the end of Phase 5, on the final selected configuration only.

Task 5: HPO Results Analysis

A list of trial scores is not analysis. The report must include:

- Optimization history plot (best metric vs trial number) showing whether the search has converged
- Hyperparameter importance plot (Optuna provides this; equivalent in other libraries)
- Parallel coordinates or slice plots showing the relationship between each hyperparameter and the metric
- A short discussion: which hyperparameters mattered most? Which had no effect? Were any results spurious (e.g., a single lucky trial)?
- The selected best configuration, with a note on how robust it is (compare to the second- and third-best configurations)

6. Integration: Bringing It Together

By Friday you should be able to describe the integrated system in one paragraph. Something like:

"Our final model uses a ResNet-50 backbone pretrained on ImageNet, with the top 6 layers fine-tuned at LR 5e-5 and a custom 3-layer classification head trained at LR 1e-3 (selected by Bayesian search over 35 trials). Class imbalance is addressed by augmenting the minority class with samples from a VAE trained on the original training set, achieving a 4-point F1 improvement over the Phase 3 model."

If you cannot summarize your system this concretely, your integration is not yet done. Spend the last day on integration, not on squeezing another 0.3% out of one workstream.

7. Refinement Report Structure

The report (12–16 pages, PDF) must contain:

1. Cover Page & Repository Link

Team, course, GitHub URL, commit hash and tag of submission, date.

2. Phase 3 Recap

Half a page. Where Phase 3 left off, what numbers you had, what you wanted to improve.

3. Transfer Learning Study

All three configurations, parameter counts, training curves, final metrics, discussion. About 2–3 pages.

4. Generative Component

Architecture, loss formulation, training curves, sample outputs, evaluation metrics, integration with main project. About 3–4 pages.

5. Hyperparameter Search Setup

Search space, strategy, budget, infrastructure. About 1 page.

6. HPO Results & Analysis

Optimization history, importance plot, parallel coordinates, written interpretation, selected configuration. About 2 pages.

7. Integrated System

The one-paragraph summary above, expanded to a full description with end-to-end diagram and final test-set-free metrics.

8. Negative Results

What did not work? A page of failed experiments and what they taught you. This is graded positively, not penalized. Honesty matters.

9. Plan for Phase 5

Specific evaluation experiments, ablations, and error analyses you will run next week.

8. Submission Guidelines

- Tag your submission commit: git tag phase4-submission && git push --tags
- Submit on the LMS: (a) PDF of refinement report, (b) link to GitHub repo, (c) commit hash, (d) link to HPO study (W&B project, exported Optuna SQLite, etc.), (e) link to generative model checkpoint
- Generative samples: include at least 16 representative samples in the report and a folder of 100+ in the repo (or linked storage)
- Late penalty: 10% per day, no submissions after 72 hours late